

GISMA

University of Applied Sciences

Advanced Databases M605

Modern Library Management System Project Report

Student Information

Full Name	Manoj Kumar
Student ID	GH1037914
Module	Advanced Databases M605A
Submission Date	18.09.2026

Project URL's

GitHub Repository	https://github.com/manojkumar-techdev/Modern-Library-Management-System/tree/main
Video Demonstration	https://drive.google.com/file/d/1Qdx0_XZ-qSXwMAWUTUX8ECVp66v3v7ZV/view?usp=sharing

Index

- 01. Introduction
- 02. Executive summary
- 03. System Design
- 04. Implementation
- 05. Results
- 06. Challenges and Solutions
- 07. Conclusion and Future Work

1. Introduction

Modern Library Management System demands a design and implementation of a detailed and automated database system that can apply either SQL and/or NoSQL technologies. The main purpose of the given project is to create a modern system including hybrid mode effectively which can handle structured, semi-structured and unstructured data to fulfill the real-world design aims.

A large number of domains can be freely opted and designed for a business or system of personal choice which can be beneficial for current world problems. The project has to follow the modern database management system including creating database, data insertion and implementation of queries and commands for performance-based optimal results.

Problem Definition

Project Requirements

- SQL/NoSQL Database
- Structured data
- Schema design
- Data insertion
- ACID compliance (e.g., user accounts, transactions)
- No full web app or user interface required

2. Executive Summary

The project Library Management System is based on using SQL (Structured Query Language) using Relational Database Management System (MySQL). This system stores information about books, members, book categories, authors, borrowing, fines and returning books in an organized way.

The core goal of this Library Management System is to automate the information using REDMS to minimize the manual actions taken in a library, improve the accuracy of library records by making it convenient for librarians to search books and members. The project offers an easy, trustworthy and optimal system for managing library operations.

Technology Used

- **Database:** MySQL
- **Language:** SQL
- **Database Design:** Relational Database Model
- **Tools:** MySQL Workbench / MySQL Server

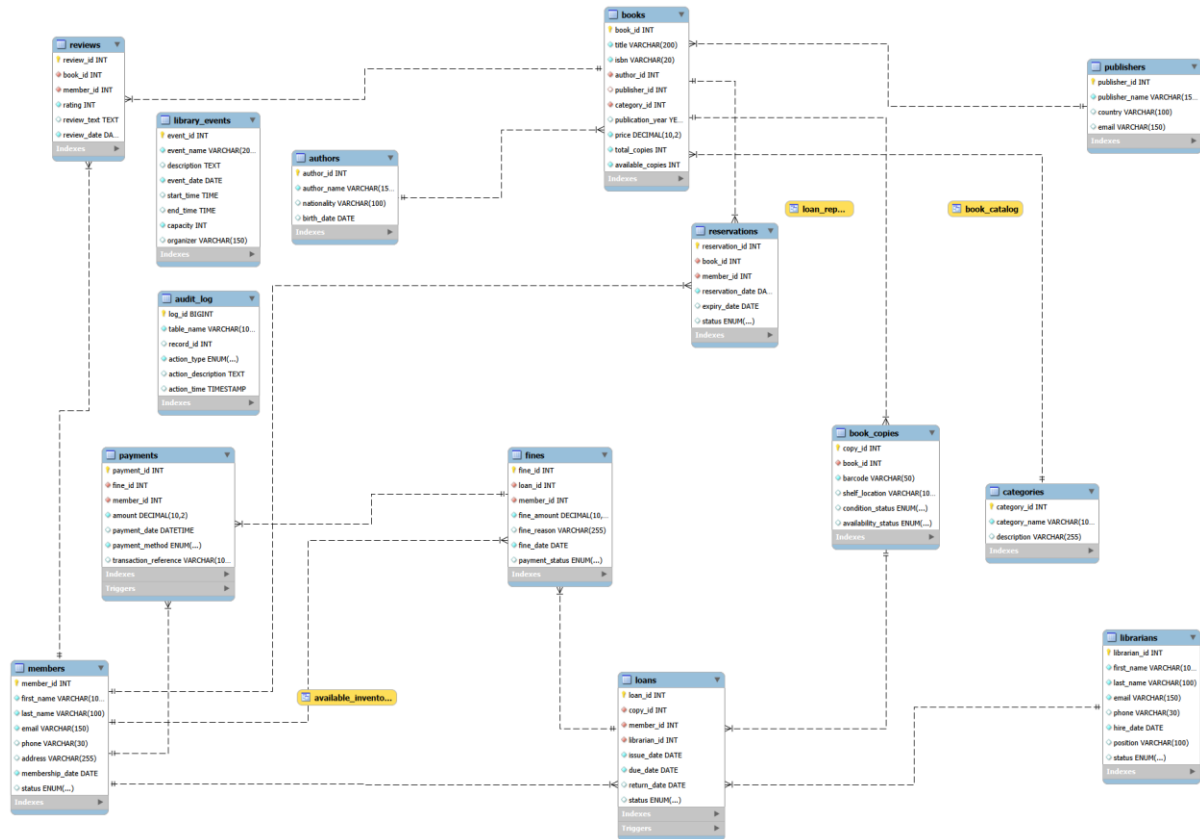
Project Features

- Database Creation
- Tables
- Primary Keys
- Foreign Keys
- Constraints
- Sample Data
- Basic Queries
- Inner, Left & Right Joins
- Aggregation
- Subqueries
- Stored Procedures
- Stored Functions
- Triggers
- Transactions
- Indexes
- Views
- Management Reports
- Inventory Management
- Reservations
- Fines & Payments
- Reviews
- Library Events
- Suppliers & Acquisitions
- Audit Logging

Project Objectives

- 1. To create a computerized system for managing library resources.
- 2. To maintain information about books and authors.
- 3. To organize books according to categories.
- 4. To maintain publisher's information.
- 5. To register and manage library members.
- 6. To manage librarians.
- 7. To record book issuing and return transactions.
- 8. To identify overdue books.
- 9. To calculate and manage library fines.
- 10. To manage book reservations.
- 11. To generate useful library reports.
- 12. To maintain data integrity using relational database constraints.

3. System Architecture



ER DIAGRAM - 1

Database Relationships

No .	Parent Table	Child Table	Primary Key	Foreign Key	Relationship
1	authors	books	authors.author_id	books.author_id	1 : N
2	publishers	books	publishers.publisher_id	books.publisher_id	1 : N
3	categories	books	categories.category_id	books.category_id	1 : N
4	books	book_copies	books.book_id	book_copies.book_id	1 : N
5	members	loans	members.member_id	loans.member_id	1 : N
6	librarians	loans	librarians.librarian_id	loans.librarian_id	1 : N
7	book_copies	loans	book_copies.copy_id	loans.copy_id	1 : N
8	books	reservations	books.book_id	reservations.book_id	1 : N
9	members	reservations	members.member_id	reservations.member_id	1 : N
10	loans	fines	loans.loan_id	fines.loan_id	1 : N
11	members	fines	members.member_id	fines.member_id	1 : N
12	fines	payments	fines.fine_id	payments.fine_id	1 : N
13	members	payments	members.member_id	payments.member_id	1 : N
14	books	reviews	books.book_id	reviews.book_id	1 : N
15	members	reviews	members.member_id	reviews.member_id	1 : N

Database Relationships — Table 1

4. Implementations

Modern Library Management System implements a wide range of features, commands and queries to make it an operational application. This application has database creation, schema design, data insertion, CRUD, triggers and views including many more components. It serves a complete digital library system for regular library functions. A librarian can perform several operations to make library management system sufficient to serve its clients by managing all library activities required to run it.

01. Database Creation
02. Table Creation
03. Data Insertion
04. Data Retrieval
05. Data Updating
06. Data Deletion
07. Table Relationships
08. Transactions
09. Stored Procedure Implementation
10. Stored Function Implementation
11. Trigger Implementation
12. View Implementation
13. Index Implementation
14. Aggregation Implementation
15. Audit Logging Implementation

5. Results

01. Database Creation

1 QUERY -- DROP DATABASE IF EXISTS library_management;

DROP DATABASE deletes the existing database if it already exists. This is useful when rerunning your project from the beginning.

```
2    -- Query 1 (Drops a Database if already a Database with same name exists)
3 •   DROP DATABASE IF EXISTS modern_library;
```

Figure 1: Example of DROP DATABASE IF EXISTS

2 QUERY -- CREATE DATABASE library_management;

CREATE DATABASE command creates a new database named library_management.

```
6    -- Query 2 (Creates a new Database)
7 •   CREATE DATABASE modern_library;
```

Figure 2: Example of CREATE DATABASE database_name

3 QUERY -- USE library_management;

USE command selects the database so that all subsequent tables and SQL objects are created inside it.

```
10   -- Query 3 (Switches to or uses a specific database mentioned in query)
11 •   USE modern_library;
```

Figure 3: Example of USE DATABASE database_name

02. Table Creation

4 Query -- CREATE TABLE authors;

The CREATE TABLE command is used to create a new table in a database. Tables store data in the form of Tabular Structure and Data is stored using table name, columns, rows, primary keys, foreign keys and constraints.

```
2    -- Query 1 (Creates table for "AUTHORS")
3 •   CREATE TABLE authors (
4       author_id INT AUTO_INCREMENT PRIMARY KEY,
5       author_name VARCHAR(150) NOT NULL,
6       nationality VARCHAR(100),
7       birth_date DATE
8   );
```

Figure 4: Example of INSERT INTO table_name

SYNTAX FOR CREATING ALL TABLES

All tables are created in Library Management System project using **CREATE TABLE** command, and tables which are created are “author, publisher, categories, books, members, librarians, book_copies, loans, reservations, fines, payments, reviews and library events tables”.

Syntax used for creating all tables:

01. **CREATE TABLE** authors ();
02. **CREATE TABLE** publisher ();
03. **CREATE TABLE** categories ();
04. **CREATE TABLE** books ();
05. **CREATE TABLE** members ();
06. **CREATE TABLE** librarians ();
07. **CREATE TABLE** book_copies ();
08. **CREATE TABLE** loans ();
09. **CREATE TABLE** reservations ();
10. **CREATE TABLE** fines ();
11. **CREATE TABLE** payments ();
12. **CREATE TABLE** reviews ();
13. **CREATE TABLE** library_events ();

03. Data Insertion

5 Query -- INSERT INTO authors

INSERT INTO command is used to insert new rows (new records) into the table. This command inserts news records for fields created during creation of a specific table.

Author table contains 100 rows of data. Similarly, all tables are inserted with 100 rows of data and data is inserted using INSERT INTO table_name. Data is inserted in “author, publisher, categories, books, members, librarians, book_copies, loans, reservations, fines, payments, reviews and library events tables”.

```
2  -- Query 1 (Inserts synthetic data into "AUTHORS" table)
3  •  INSERT INTO authors
4      (author_name, nationality, birth_date)
5      VALUES
6      ('William Shakespeare', 'English', '1564-04-23'),
7      ('Jane Austen', 'English', '1775-12-16'),
8      ('Mark Twain', 'American', '1835-11-30'),
9      ('Charles Dickens', 'English', '1812-02-07'),
```

Figure 5: Example of INSERT INTO table_name

Syntax used for inserting data in all tables:

```
INSERT INTO authors () VALUES ();
INSERT INTO publishers () VALUES ();
INSERT INTO categories () VALUES ();
INSERT INTO books () VALUES ();
INSERT INTO members () VALUES ();
INSERT INTO book_copies () VALUES ();
INSERT INTO librarians () VALUES ();
INSERT INTO loans () VALUES ();
INSERT INTO reservations () VALUES ();
INSERT INTO fines () VALUES ();
INSERT INTO payments () VALUES ();
INSERT INTO reviews () VALUES ();
INSERT INTO library_events () VALUES ();
```

04. Basic & CRUD Queries

6 Query -- Display all books (SELECT * FROM books);

To display all books from book table, SELECT * FROM books query is used. This query use “*” which means all. So when SELECT * is given, it selects all books from table “book” and shows a table containing all books along with columns used at the time of table creation.

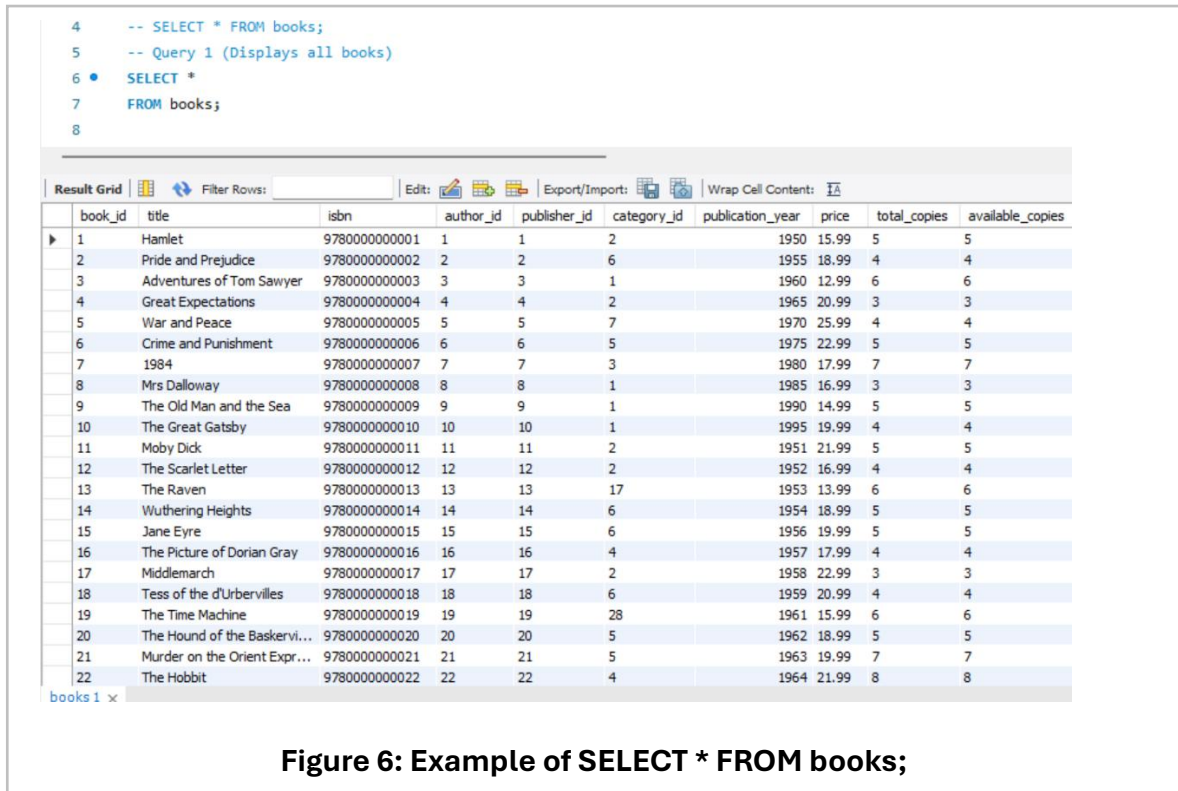
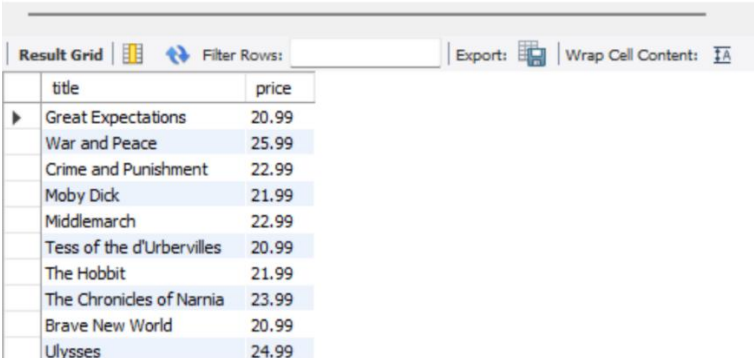


Figure 6: Example of SELECT * FROM books;

7 Query -- Display all books (SELECT * FROM books WHERE price > 20);

WHERE clauses are used to select the books from table using logic greater than filter books which have price more than 20. WHERE selects, updates, or deletes the rows with the help of operators =, <, >, <=, >=, <>, AND, OR

```
10  -- SELECT * FROM books WHERE price > 20;
11  -- Query 2 (Displays books costing more than 20)
12  •  SELECT
13      title,
14      price
15  FROM books
```



The screenshot shows a database query result grid. The grid has two columns: 'title' and 'price'. The data is as follows:

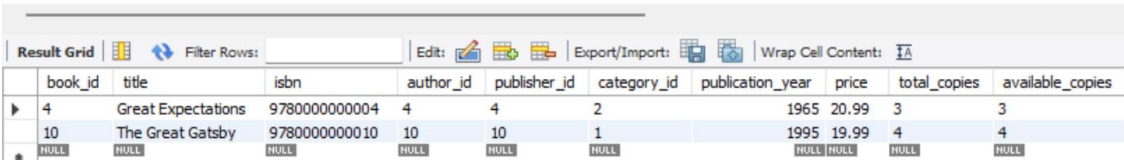
title	price
Great Expectations	20.99
War and Peace	25.99
Crime and Punishment	22.99
Moby Dick	21.99
Middlemarch	22.99
Tess of the d'Urbervilles	20.99
The Hobbit	21.99
The Chronicles of Narnia	23.99
Brave New World	20.99
Ulysses	24.99

Figure 7: Example of SELECT * FROM books WHERE price > 20

8 Query -- Display all books (SELECT * FROM books WHERE title LIKE '%Great%');

Like clause is used to search for a specific pattern in text and it is commonly used with WHERE clause. In the example, **LIKE** clause searched for “Great” keyword and it displayed the books with title including “Great”. It searched for **2 books** in the entire Database.

```
19  -- WHERE title LIKE '%Great%';
20  -- Query 3 (finds all books whose title contains the word/letters "Great")
21  •  SELECT *
22  FROM books
23  WHERE title LIKE '%Great%';
24
```



The screenshot shows a database query result grid. The grid has ten columns: 'book_id', 'title', 'isbn', 'author_id', 'publisher_id', 'category_id', 'publication_year', 'price', 'total_copies', and 'available_copies'. The data is as follows:

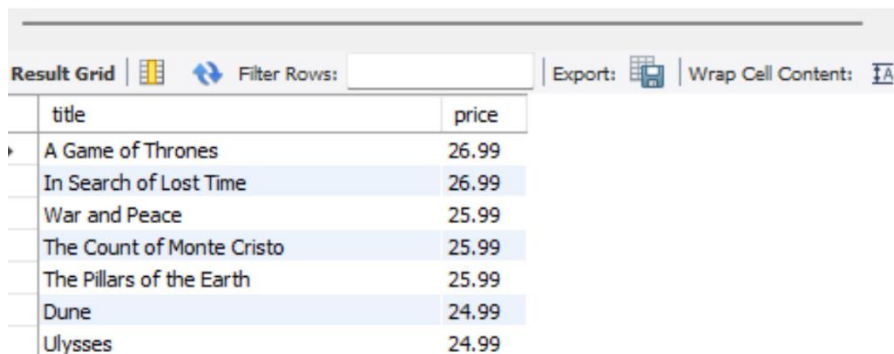
book_id	title	isbn	author_id	publisher_id	category_id	publication_year	price	total_copies	available_copies
4	Great Expectations	9780000000004	4	4	2	1965	20.99	3	3
10	The Great Gatsby	9780000000010	10	10	1	1995	19.99	4	4

Figure 8: Example of SELECT * FROM books WHERE title LIKE '%Great%'

9 Query -- Sort books (ORDERED BY price DESC;)

ORDERED BY and **DESC** works together to filter all books and order them in a descending order. But descending is applied on a filter value given to query. Price is used as a filter value which is compared in Descending order and all books are listed from top to bottom in a decreasing price order.

```
26      -- ORDERED BY price DESC;
27      -- Query 4 (Sorts book by title)
28 •    SELECT
29          title,
30          price
31      FROM books
32      ORDER BY price DESC;
33
```



The screenshot shows a database interface with a 'Result Grid' tab. Above the grid, there are icons for 'Filter Rows', 'Export', and 'Wrap Cell Content'. The grid displays the results of the query, with columns 'title' and 'price'. The books are sorted by price in descending order.

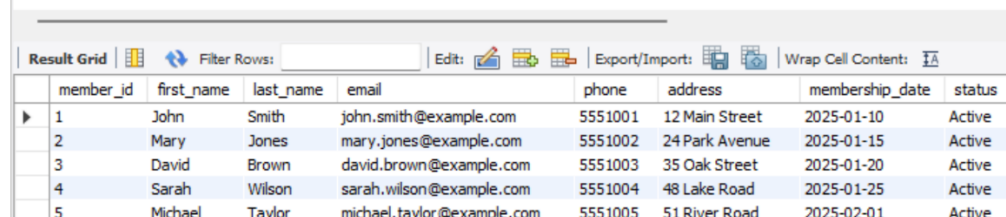
	title	price
▶	A Game of Thrones	26.99
	In Search of Lost Time	26.99
	War and Peace	25.99
	The Count of Monte Cristo	25.99
	The Pillars of the Earth	25.99
	Dune	24.99
	Ulysses	24.99

Figure 9: Example of ORDERED BY with DESC

10 Query -- Equality checks for status (WHERE status = 'Active';)

WHERE and **=** operator together displays the status of members where they are active. **SELECT *** selects all columns and FROM members use the members' table to compare **WHERE** status is 'Active' in the records. This query shows only where the status is exactly active.

```
35      -- WHERE status = 'Active';
36      -- Query 5 (Displays active members)
37 •    SELECT *
38      FROM members
39      WHERE status = 'Active';
40
```



The screenshot shows a database interface with a 'Result Grid' tab. Above the grid, there are icons for 'Filter Rows', 'Edit', 'Export/Import', and 'Wrap Cell Content'. The grid displays the results of the query, with columns 'member_id', 'first_name', 'last_name', 'email', 'phone', 'address', 'membership_date', and 'status'. All members listed are 'Active'.

	member_id	first_name	last_name	email	phone	address	membership_date	status
▶	1	John	Smith	john.smith@example.com	5551001	12 Main Street	2025-01-10	Active
	2	Mary	Jones	mary.jones@example.com	5551002	24 Park Avenue	2025-01-15	Active
	3	David	Brown	david.brown@example.com	5551003	35 Oak Street	2025-01-20	Active
	4	Sarah	Wilson	sarah.wilson@example.com	5551004	48 Lake Road	2025-01-25	Active
	5	Michael	Taylor	michael.taylor@example.com	5551005	51 River Road	2025-02-01	Active

Figure 10: Example of WHERE with “=” Operator

11 Query – CREATE (Adds a new library member)

```
55 -- CREATE: Add a new library member
56 • INSERT INTO members
57 (first_name, last_name, email, phone, address, membership_date, status)
58 VALUES
59 ('John', 'Smith', 'john.smith@library.com',
60 '491900000001', 'Berlin, Germany', '2026-09-17', 'Active');
--
```

Figure 11: Example of CREATE

12 Query – READ (Displays a specific member)

```
80 # Query -- 11
81 -- READ: Displays a specific member
82 • SELECT *
83 FROM members
84 WHERE email = 'john.smith@library.com';
```

Figure 12: Example of READ

13 Query – UPDATE (Changes the member's phone number)

```
88 -- @ Query -- 12
89 -- UPDATE: Change the member's phone number
90 • UPDATE members
91 SET phone = '491900000999'
92 WHERE email = 'john.smith@library.com';
```

Figure 13: Example of UPDATE

14 Query – DELETE (Removes a member where email is mentioned)

```
95 -- @ Query -- 13
96 -- DELETE: Remove the member
97 • DELETE FROM members
98 WHERE email = 'john.smith@library.com';
```

Figure 14: Example of DELETE

05. Joins

JOIN SCOPE FOR PROJECT:

- **INNER JOIN**
- **LEFT JOIN**
- **RIGHT JOIN**
- **MULTIPLE JOINS**

15 Query -- INNER JOIN (multi-table INNER JOIN query)

INNER JOIN combines information from four tables: books, authors, publishers, and categories. **INNER JOIN** returns only records that have matching data in all joined tables. If a book has no matching author, that book will not appear in the result.

This query first selects columns, then connects books with authors, then connects each book to its publisher and connects each book to its category. So, multiple **INNER JOINS** work one after another to produce the result.

```
5      -- Query 1 (Inner join)
6      -- Shows books with Author, Publisher, and Category
7  •   SELECT
8      b.book_id,
9      b.title,
10     a.author_name,
11     p.publisher_name,
12     c.category_name
13 FROM books b
14 INNER JOIN authors a
15     ON b.author_id = a.author_id
16 INNER JOIN publishers p
17     ON b.publisher_id = p.publisher_id
18 INNER JOIN categories c
19     ON b.category_id = c.category_id;
```

Figure 15: Example of INNER JOIN

06. Aggregations

AGGREGATION SCOPE FOR PROJECT:

- **COUNT ()**
- **SUM ()**
- **AVG ()**
- **MAX ()**
- **MIN ()**

16 QUERY -- Aggregation (COUNT(*))

Aggregation command is used to run a query to count the total number of rows. The book table contains 100 rows. When the query is run, **COUNT(*)** counts all rows, and AS total_titles, gives total_titles. FROM books counts rows from the books table.

```
5      -- Query 1 (Displays total number of titles)
6 •    SELECT COUNT(*) AS total_titles
7      FROM books;
```

Figure 16: Example of COUNT aggregation

07. Stored Procedures

STORED PROCEDURE SCOPE FOR PROJECT:

- CREATE PROCEDURE SearchBooks
- CREATE PROCEDURE GetMemberLoans

17 QUERY – Stored Procedure (Searches books)

This stored procedure is **correct and compatible** with books and authors tables.

```
4  -- Query 1 (Searches for books containing Great in the title or author name)
5  -- DROP PROCEDURE IF EXISTS SearchBook(Runs the query again if SearchBooks Procedure already exists.
6  -- If DROP PROCEDURE not used, MySQL shows that procedure already exists.
7  • DROP PROCEDURE IF EXISTS SearchBooks;
8
9  DELIMITER //
10
11 • CREATE PROCEDURE SearchBooks(
12     IN p_keyword VARCHAR(100)
13 )
14 BEGIN
15     SELECT
16         b.book_id,
17         b.title,
18         a.author_name,
19         b.price,
20         b.available_copies
21     FROM books b
22     JOIN authors a
23         ON b.author_id = a.author_id
24     WHERE b.title LIKE CONCAT('%', p_keyword, '%')
25         OR a.author_name LIKE CONCAT('%', p_keyword, '%');
26 END //
27
28 DELIMITER ;
29
30 • CALL SearchBooks('Harry');
```

Figure 17: Example of STORED PROCEDURE

08. Stored Function Implementation

18 Query: This stored function is called CalculateFine, and it calculates a library fine at 1.00 per overdue day.

Working procedure:

If the book has been returned, it calculates:

return_date - due_date

If the book has not been returned, it calculates:

current date - due_date

If the result is zero or negative, the fine is 0.00.

Otherwise, the fine is **2.00 per late day**.

```
4  -- Query 1 (Calculates fine and charges 2.00 per overdue day)
5  DELIMITER //
6  • CREATE FUNCTION CalculateFine(
7      p_due_date DATE,
8      p_return_date DATE
9  )
10     RETURNS DECIMAL(10,2)
11     NOT DETERMINISTIC
12     BEGIN
13
14         DECLARE v_late_days INT;
15
16         IF p_return_date IS NULL THEN
17
18             SET v_late_days =
19                 DATEDIFF(CURDATE(), p_due_date);
20
21         ELSE
22
23             SET v_late_days =
24                 DATEDIFF(p_return_date, p_due_date);
25
26         END IF;
27
28         IF v_late_days <= 0 THEN
29             RETURN 0.00;
30         END IF;
31
32         RETURN v_late_days * 2.00;
33
34     END //
35  DELIMITER ;
37  • SELECT
38      loan_id,
39      due_date,
40      return_date,
41      CalculateFine(
42          due_date,
43          return_date
44      ) AS calculated_fine
45  FROM loans;
```

Figure 18: Example of STORED FUNCTIONS

09. Trigger Implementation

19 Query – Prevents a book to issue if a copy is not available.

This Trigger runs before a new loan is inserted, after that gets the availability status of the requested copy. In the next step, it checks if the copy is available. If copy is not available, it stops the insert with an error.

```
4  -- Query 1 (prevent a library book copy from being borrowed when it is not available)
5  DELIMITER //
6  • CREATE TRIGGER before_loan_insert
7    BEFORE INSERT ON loans
8    FOR EACH ROW
9    BEGIN
10
11      DECLARE v_status VARCHAR(20);
12
13      SELECT availability_status
14      INTO v_status
15      FROM book_copies
16      WHERE copy_id = NEW.copy_id;
17
18      IF v_status <> 'Available' THEN
19
20        SIGNAL SQLSTATE '45000'
21        SET MESSAGE_TEXT =
22          'This book copy is not available for borrowing';
23
24      END IF;
25
26  END //
27  DELIMITER ;
```

Figure 19: Example of TRIGGER

10. Transactions

20 Query - Records a €5.00 card payment.

This transaction checks fine id → 5, member id 5, amount → 5, payment_method → card and transaction_reference → TXN-20260915 and finally commit permanent changes in payments. It results in 5.00 which is valid output for this transaction.

```
5      # -- Query 1 (Start Transaction)
6      -- (Begins a transaction, which groups one or more SQL operations into a single unit)
7      -- Example Query (This transaction records a $5.00 card payment)
8
9      •   START TRANSACTION;
10     •   INSERT INTO payments (
11         |       fine_id,
12         |       member_id,
13         |       amount,
14         |       payment_method,
15         |       transaction_reference
16         |   )
17     ○   VALUES (
18         |       5,
19         |       5,
20         |       5.00,
21         |       'Card',
22         |       'TXN-20260915-005'
23         |   );
24     •   COMMIT;
```

Figure 20: Example of TRANSACTION

11. Report

21 Query -- Displays library dashboard summary query

```
4  -- Query 1 (Displays library dashboard)
5  • SELECT
6
7  (
8      SELECT COUNT(*)
9      FROM books
10 ) AS total_titles,
11
12 (
13     SELECT COUNT(*)
14     FROM book_copies
15 ) AS physical_copies,
16
17 (
18     SELECT COUNT(*)
19     FROM book_copies
20     WHERE availability_status = 'Available'
21 ) AS available_copies,
22
23 (
24     SELECT COUNT(*)
25     FROM book_copies
26     WHERE availability_status = 'Borrowed'
27 ) AS borrowed_copies,
28
29 (
30     SELECT COUNT(*)
31     FROM members
32     WHERE status = 'Active'
33 ) AS active_members,
34
35 (
36     SELECT COUNT(*)
37     FROM loans
38     WHERE status = 'Issued'
39 ) AS active_loans,
40
41 (
42     SELECT COUNT(*)
43     FROM loans
44     WHERE status = 'Overdue'
45 ) AS overdue_loans,
46
47 (
48     SELECT COALESCE(
49         SUM(fine_amount), 0
50     )
51     FROM fines
52     WHERE payment_status <> 'Paid'
53 ) AS unpaid_fines;
```

Figure 21: Example of REPORT

Result Grid Filter Rows: Export: Wrap Cell Content:								
	total_titles	physical_copies	available_copies	borrowed_copies	active_members	active_loans	overdue_loans	unpaid_fines
▶	100	100	100	0	100	37	15	737.00

Figure 22: Result

12. Inventory Management

22 Query – Displays all currently available physical book copies.

```
18  -- Query 2 (Displays available physical copies)
19  •  SELECT
20      bc.barcode,
21      b.title,
22      bc.shelf_location,
23      bc.condition_status
24  FROM book_copies bc
25  JOIN books b
26      ON bc.book_id = b.book_id
27  WHERE bc.availability_status = 'Available';
```

Figure 23: Example of Inventory Management

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
barcode	title	shelf_location	condition_status
BC000001	Hamlet	A-01	Excellent
BC000002	Pride and Prejudice	A-01	Good
BC000003	Adventures of Tom Sawyer	A-02	Good
BC000004	Great Expectations	A-02	Excellent
BC000005	War and Peace	A-03	Good
BC000006	Crime and Punishment	A-03	Good
BC000007	1984	A-04	Excellent
BC000008	Mrs Dalloway	A-04	Good
BC000009	The Old Man and the Sea	A-05	Good
BC000010	The Great Gatsby	A-05	Excellent
BC000011	Moby Dick	B-01	Good
BC000012	The Scarlet Letter	B-01	Excellent
BC000013	The Raven	B-02	Good
BC000014	Wuthering Heights	B-02	Good
BC000015	Jane Eyre	B-03	Excellent
BC000016	The Picture of Dorian Gray	B-03	Good

Figure 24: Result

13. Reservations

23 Query – Creates new reservation with reservation_date → current date
expiry_date → current date + 7 days status → Pending Status.

```
32 -- Query 2 (Displays created reservations)
33 • INSERT INTO reservations
34 (
35     book_id,
36     member_id,
37     reservation_date,
38     expiry_date,
39     status
40 )
41 VALUES
42 (
43     7,
44     2,
45     CURDATE(),
46     DATE_ADD(
47         CURDATE(),
48         INTERVAL 7 DAY
49     ),
50     'Pending'
51 );
```

Figure 25: Example of Reservation

14. Fines & Payments

24 Query - Displays a useful unpaid/partially paid fines report.

```
4 -- Query 1 (Displays outstanding fines)
5 • SELECT
6     f.fine_id,
7
8     CONCAT(
9         m.first_name,
10        ' ',
11        m.last_name
12    ) AS member_name,
13
14     f.fine_amount,
15     f.fine_reason,
16     f.fine_date,
17     f.payment_status
18
19 FROM fines f
20
21 JOIN members m
22     ON f.member_id = m.member_id
23
24 WHERE f.payment_status <> 'Paid'
25
26 ORDER BY f.fine_amount DESC;
```

Figure 26: Example of Fines & Payments

Result Grid						
		Filter Rows:	Export:	Wrap Cell Content:		
	fine_id	member_name	fine_amount	fine_reason	fine_date	payment_status
▶	26	Eva Wallace	25.00	Book overdue	2026-09-28	Unpaid
	18	Claire Griffin	22.00	Book overdue	2026-09-18	Unpaid
	14	Brandon Sanders	21.00	Book overdue	2026-07-22	Unpaid
	95	Tyler Jenkins	21.00	Book overdue	2026-08-11	Unpaid
	8	Nicholas Nelson	20.00	Book overdue	2026-04-25	Unpaid
	87	Matthew Murphy	20.00	Book overdue	2026-06-11	Unpaid
	25	Logan Sullivan	20.00	Book overdue	2026-09-27	Unpaid
	99	Isaac Bryant	20.00	Book overdue	2026-09-11	Unpaid
	12	Joseph Richardson	19.00	Book overdue	2026-06-22	Unpaid

Figure 27: Result

15. Library Events

25 Query – Displays upcoming library events, including events happening today.

```

3      -- Query 1 (Displays upcoming events)
4      •  SELECT
5          event_id,
6          event_name,
7          description,
8          event_date,
9          start_time,
10         end_time,
11         capacity,
12         organizer
13     FROM library_events
14     WHERE event_date >= CURDATE()
15     ORDER BY event_date, start_time;

```

Figure 28: Example of Library Events

Result Grid		Filter Rows:		Export:	Wrap Cell Content:			
	event_id	event_name	description	event_date	start_time	end_time	capacity	organizer
▶	27	Health Awareness Day	Health and wellness awareness program	2026-09-20	10:00:00	15:00:00	150	Health Department
	28	Environmental Seminar	Discussion on environmental sustainability	2026-09-25	14:00:00	17:00:00	70	Environment Department
	29	Book Donation Drive	Community book donation program	2026-10-01	09:00:00	16:00:00	200	Community Outreach
	30	Reading Marathon	Full day community reading event	2026-10-05	09:00:00	18:00:00	120	Central Library
	31	Poetry Competition	Poetry writing and performance competition	2026-10-10	13:00:00	17:00:00	80	Literature Department
	32	Short Story Contest	Short story writing competition	2026-10-15	10:00:00	16:00:00	70	Writing Department
	33	Library Technology Day	Technology demonstrations for members	2026-10-20	11:00:00	15:00:00	100	IT Department
	34	Cybersecurity Awareness	Cybersecurity awareness and safety session	2026-10-25	14:00:00	17:00:00	60	IT Department
	35	Artificial Intelligence Seminar	Introduction to artificial intelligence	2026-11-01	10:00:00	13:00:00	90	Technology Department
	36	Data Science Workshop	Introduction to data science	2026-11-05	13:00:00	17:00:00	45	IT Department
	37	Programming Workshop	Beginner programming workshop	2026-11-10	10:00:00	14:00:00	50	Computer Science Team
	38	Database Seminar	Database design and management seminar	2026-11-15	14:00:00	17:00:00	55	IT Department
	39	Web Development Workshop	Introduction to web development	2026-11-20	10:00:00	14:00:00	50	Technology Department

Figure 29: Result

16. Indexes

26 Query -- Creates an index named `idx_books_title` on the `title` column of the `books` table. Index is used to improve queries that search or sort by book title.

```
4  -- Query 1 (Creates an Index on Book Titles)
5  • CREATE INDEX idx_books_title
6  ON books(title);
```

Figure 30: Example of Indexes

4. Challenges and Solutions

Challenge 1 – There was a schema mismatch for member table.

Member table had columns:

```
status ENUM('Active', 'Inactive', 'Suspended') DEFAULT 'Active'
```

Figure 31: Example of Schema Mismatch Error

Query used “`membership_status`”

When column does not exist, MySQL returns an error such as Unknown column 'membership_status'

Corrected Query Used Afterwards is:

```
85 • INSERT INTO members
86     (first_name, last_name, email, phone, address, membership_date, status)
87     VALUES
88     ('John', 'Smith', 'john.smith@library.com',
89     '491900000001', 'Berlin, Germany', '2026-09-17', 'Active');
```

Figure 32: Solution to Schema Mismatch

Challenge 2 – Use of **Deterministic** vs **Not Deterministic** in stored functions.

Stored Function with Error:

```
4  -- Query 1 (Calculates fine and charges 2.00 per overdue day)
5  DELIMITER //
6  CREATE FUNCTION CalculateFine(
7      p_due_date DATE,
8      p_return_date DATE
9  )
10 RETURNS DECIMAL(10,2)
11 DETERMINISTIC
12 BEGIN
```

Figure 33: Example of Deterministic Use

Solution: If a function relies on the current date/time, random values, or other changing data, function must not be declare it DETERMINISTIC. So, in such condition, Not Deterministic has to be used.

```
4  -- Query 1 (Calculates fine and charges 2.00 per overdue day)
5  DELIMITER //
6  CREATE FUNCTION CalculateFine(
7      p_due_date DATE,
8      p_return_date DATE
9  )
10 RETURNS DECIMAL(10,2)
11 NOT DETERMINISTIC
12 BEGIN
```

Figure 44: Solution to Deterministic Use

4. Conclusion and Future Work

Modern Library Management System implements all the necessary guidelines of module M604:

Backend Development: using SQL Language and MySQL Database Software.

Data Interaction: Database, tables, aggregation, views, transactions and queries are functional.

System Design: ER diagram is the visual representation of Relationships existing in Database.

Implementations:

Database is created to store the entire data of Modern Library Management System.

Tables are created using CREATE TABLE for different components of the Applications. Tables contain columns and rows to store data along with constraints, keys and indexes.

Data is inserted in relevant tables using INSERT commands, and structure of data is built as per the TABLES designed for Library Management System.

The applications works without any errors and produce the desired results as per the queries inserted and data can be seen in the tables in result section of MySQL GUI.

Some challenges have been faced during the application development, and those errors have been eliminated by depth work and system changes with the help of structure of storing tables and query commands.

Some improvements can be made in the future. Front-end, APIs or Advanced Backend System can be developed using File Automation, Advance Searches, Online Reservation, Notification System, Web or Mobile Interface Integration.

References

1. MySQL Documentation: <https://dev.mysql.com/doc/>
2. MySQL 8.0 Reference Manual: <https://dev.mysql.com/doc/refman/8.0/en/>
3. W3Schools – SQL Tutorial: <https://www.w3schools.com/sql/>
4. W3Schools – MySQL Tutorial: <https://www.w3schools.com/mysql/>
5. GeeksforGeeks – DBMS and SQL: <https://www.geeksforgeeks.org/>
6. A Guide to MySQL: Pratt, P. J., & Last, M. Z. (2005). Publisher: Cengage Learning. Publication date: December 9, 2005.
7. MySQL Bible : Suehring, S. (2002). Publisher: Wiley. Publication date: July 2002.
8. Database System Concepts (7th ed.). Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Publisher: McGraw-Hill Education. Publication year: 2019.